RESEARCH MEETING

# Helix: Serving Large Language Models over Heterogeneous GPUs and Network via Max-Flow

## Joint Model Placement and Per-Request Scheduling

**Yu-Jui Liang**
Advisor: Hsiang-Yun Cheng
National Taiwan University | Academia Sinica | 27 Oct 2026

---

Paper: Mei et al., ASPLOS 2025. Local PDF is the sole factual source.

# Presentation Roadmap



1. **Background / Motivation**
   Why mixed GPUs and wide-area links break uniform pipelines.
2. **Methodology**
   Max-flow placement, MILP constraints, and per-request pipelines.
3. **Experimental Results**
   Single-cluster prototype, plus simulated geo-distributed and 42-node runs.
4. **Discussion / Conclusion**
   What the ratios do and do not say, next to ThunderServe, Hetis, and Tessera.

Source: ASPLOS '25 Helix, local PDF. The comparison page is our synthesis.

# Why Heterogeneous and Geo-Distributed Serving?



**Table 3.** Datasheet properties. SXM for H100 and A100. Not measured serving throughput.

| GPU | FP16 (TFLOP/s) | Memory | Bandwidth | Price (USD) |
|---|---|---|---|---|
| H100 | 1979 | 80 GB | 3350 GB/s | 25k–40k |
| A100 | 312 | 40 GB | 1555 GB/s | 10k–15k |
| L4 | 242 | 24 GB | 300 GB/s | ∼3k |
| T4 | 65 | 16 GB | 300 GB/s | ∼1k |

- Table 1: if half the memory stores parameters, LLaMA-2 70B needs 12 L4s, 7 A100s, or 4 H100s.
- Table 4: in one us-east window the quota was 8 A100 + 8 L4 + 16 T4, but only 16 T4s were deployed.
- The paper's claim: eight L4s have FP16 throughput comparable to one H100, with more memory and less power.

**Scarcity, not a universal speed ranking, is why Helix aggregates mixed GPUs across regions.**

Source: Sec. 1, Tables 1–4. Eight-L4 comparison is a datasheet argument.

# Parallelism Across Nodes and GPUs

HELIX / BACKGROUND

### TENSOR PARALLELISM

**Inside one node**

Each layer needs two collectives. Helix does not use TP across the high-latency network.

### PETALS

**Decentralized, greedy**

A new machine takes the least-loaded stage. There is no global plan for a known cluster.

### PIPELINE PARALLELISM

**Across nodes**

Only the activation crosses a stage boundary. Equal cuts leave pipeline bubbles on slow GPUs.

### SWARM

**Even cuts, local routing**

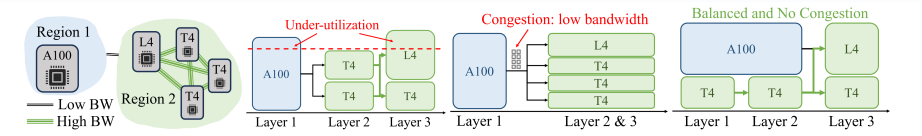
The model is partitioned evenly. The next stage is chosen from real-time throughput. It is a training method, reused here as a baseline.

**Helix keeps TP inside a node and uses PP across nodes, then refuses to cut the model into equal stages.**

Source: Sec. 2.2 and Sec. 3. Swarm is Ryabinin et al.; Petals is Borzunov et al.

# Uniform Model Placement Is Sub-Optimal

HELIX / MOTIVATION




**(a)** Five nodes in two clusters. **(b)** Uniform model partition. **(c)** Balanced FLOPs. **(d)** Network aware placement.

**Figure 1.** Examples of sub-optimal model placement and request schedule. 1a) all GPUs and network condition in this example. The order of compute capacity is: A100 > L4 > T4; 1b) Model placement by uniformly partition the model, then allocate devices by a balanced compute capacity; 1c) Co-optimizing model partition and device placement to make the compute capacity more balanced; 1d) Co-optimizing model partition, device placement, and request scheduling in a network-aware way.

Fig. 1a–c. Illustrative example, not the 24-node cluster. Order: A100 > L4 > T4.

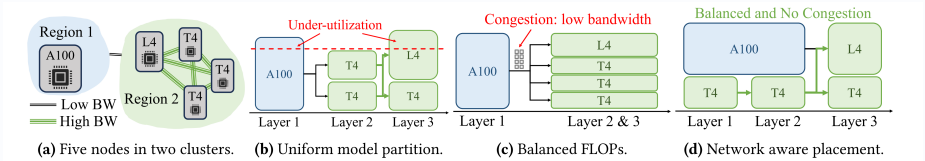
(b) Equal cuts still wait on the slower stage. (c) Balanced FLOPs still cross the slow inter-region link on every request.

**A balanced FLOP cut is not a maximum-throughput serving plan.**

Source: Sec. 3.1 and Sec. 4.2, Fig. 1a–c. PDF p. 4.

# Network-Unaware Routing Creates Congestion






**(a)** Five nodes in two clusters. **(b)** Uniform model partition. **(c)** Balanced FLOPs. **(d)** Network aware placement.

**Figure 1.** Examples of sub-optimal model placement and request schedule. 1a) all GPUs and network condition in this example. The order of compute capacity is: A100 > L4 > T4; 1b) Model placement by uniformly partition the model, then allocate devices by a balanced compute capacity; 1c) Co-optimizing model partition and device placement to make the compute capacity more balanced; 1d) Co-optimizing model partition, device placement, and request scheduling in a network-aware way.

Fig. 1c–d. Same illustrative cluster as the previous slide.

- In (c) the A100's send side congests: every activation still crosses into Region 2.
- (d) runs two T4s beside the A100, so less traffic uses the slow link.
- Partition, placement, and routing have to be chosen together.

**Network capacity is part of placement, not a detail added after the GPUs are assigned.**

Source: Sec. 4.2, Fig. 1c–d. PDF p. 4.

# Helix Design Requirements

HELIX / PROBLEM

### 1 / PLACEMENT

**One plan per cluster**

Assign each node a contiguous layer range that respects GPU memory and the links it will use.

### 2 / ESTIMATE

**Tokens per second**

Score a placement by how many tokens can move from the coordinator, through the GPUs, and back.

### 3 / SCHEDULE

**One path per request**

Do not freeze a few disjoint pipelines. Let requests share nodes when the flow says they should.

- Offline placement and online scheduling are different timescales. They share one graph.
- KV-cache capacity is not a third MILP objective. It is enforced later by masking nodes.

**Helix separates a slow placement search from a fast per-request routing decision.**

Source: Sec. 3 and Sec. 5. The KV split is the paper's design, stated across Sec. 4.4 and Sec. 5.2.

# Helix System Overview

HELIX / SYSTEM

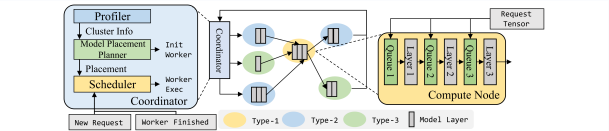



**Figure 3. Helix overview.** In Helix, the coordinator plans model placement as described in Sec. 4.4. We only need to run model placement once for each cluster. When a new request arrives, the coordinator node runs Helix scheduler to assign it a per-request pipeline and sends it to the first node in the pipeline. Each compute node in the pipeline performs inference on the request on the layers it is responsible for and sends the (output for the) request to the next node in the pipeline. When the last node in the pipeline finishes performing inference on its layers, it will send the output token for the request to the coordinator (Worker Finished). The coordinator schedules generation of the next token for the request using the same pipeline.

Fig. 3. Coordinator plans once; workers execute the assigned layers.

- Profiler collects cluster info. The planner solves placement once per cluster.
- A new request gets a pipeline and is sent to the first node only.
- Each node runs its layers and forwards the activation.
- The last node returns the token. The next token reuses the same pipeline.

**The coordinator owns placement and scheduling. Compute nodes do not re-plan the route.**

Source: Sec. 5, Fig. 3. PDF p. 7.

# From Model Placement to a Flow Network

HELIX / FORMULATION

- A cluster has one coordinator $h$ and compute nodes $C$. A multi-GPU machine is one logical node: compute and VRAM are summed.
- Placement $\Psi$ gives each node a contiguous layer range. Scheduling $\phi$ gives each request an ordered node sequence.
- A common score keeps only the slowest stage, $\min_i \sum_j \text{capacity}(j) \cdot \mathbf{1}[i \in \Psi(j)]$.
- Fig. 1c has balanced stage FLOPs and is still limited by the wide-area link. The min-stage score cannot see that.

**Helix scores a placement by the max flow of a graph, in tokens per second.**

Source: Sec. 4.1–4.2. The min-stage formula is the paper's description of prior heuristics.

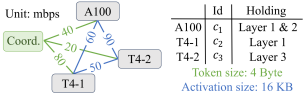
# Max-Flow Graph Abstraction



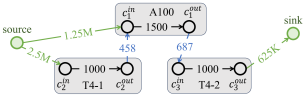
- Each compute node becomes $c^{\text{in}} \rightarrow c^{\text{out}}$.
- The coordinator becomes the source and the sink.
- An edge capacity is how many tokens may cross it per second.
- Max flow from source to sink is the serving throughput of this placement.

**The graph is an accounting device for one fixed placement.**

Source: Sec. 4.3, Fig. 2. PDF p. 4.




**(a)** A 3-node cluster with model placement. Network connections between the coordinator and compute nodes transmit tokens (4 Byte) while others transmit intermediate activations (16 KB)




**(b)** Graph abstraction of the cluster.

**Figure 2.** Graph abstraction of a 3-node cluster with given model placement. Numbers on the edges in Fig. 2b represent their capacity, which is the number of tokens that can pass through the edges per second. Max flow between source and sink equals the max serving throughput of the cluster.

Fig. 2. Capacities are tokens/s.

# Modeling Compute Capacity

HELIX / FORMULATION

### NODE EDGE

$c^{\text{in}} \rightarrow c^{\text{out}}$

Capacity is the minimum of profiled compute throughput and the node's network throughput.

### LAYER COUNT

**$T_j$ tokens/s**

$T_j$ is the profiled rate when that node holds $j$ layers. It becomes the MILP compute limit.

### PROFILING

**Once per node**

Helix measures throughput rather than using the datasheet TFLOP/s from Table 3.

### NOT A GPU

**Source and sink**

The coordinator injects tokens and collects tokens. It is not given a compute edge.

**Compute capacity is a profiled token rate, and it already takes the node's network limit into account.**

Source: Sec. 4.3. $T_j$ is defined with Constraint 3 in Sec. 4.4.

# Modeling Network Capacity

HELIX / FORMULATION

A link is valid only in one of three cases:

1. Coordinator $\rightarrow$ a node that holds the first layer. Payload is a token (Fig. 2 uses 4 bytes).
2. A node that holds the last layer $\rightarrow$ coordinator. Payload is again a token.
3. Node $i \rightarrow$ node $j$ when $j$ holds the layers needed immediately after $i$. Payload is an activation (Fig. 2 uses 16 KB).

$$\text{capacity (tokens/s)} = \frac{\text{profiled average bandwidth}}{\text{bytes per token or activation}}.$$

The formula is the paper's rule. The byte sizes above are the Fig. 2 annotations, not universal constants.

**Activations cost far more tokens of budget than the final output token, so wide-area links saturate first.**

Source: Sec. 4.3. Bandwidth is a one-time average, not a runtime measurement.

# Why Max Flow Represents Serving Throughput

HELIX / FORMULATION

- One unit of flow is one token that can pass a node or a link in one second.
- Conservation forces tokens into $c^{\text{in}}$ to leave through $c^{\text{out}}$.
- Helix computes the source–sink max flow with preflow-push. That value is the placement's max serving throughput.
- The model is a steady token rate. It does not output prompt latency, decode latency, or a percentile.
- A later early-stop bound is the sum of node compute rates, divided by the number of layers.

**Max flow answers "how many tokens/s can this placement carry?" It is not a latency model.**

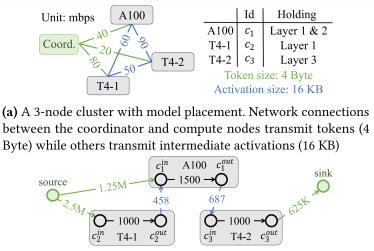
Source: Sec. 4.3 and the early-stop remark in Sec. 4.5.

# Worked Three-Node Example



- A100 holds layers 1–2. T4-1 holds layer 1. T4-2 holds layer 3.
- Node edges are marked 1500, 1000, and 1000 tokens/s.
- Token-sized edges are marked 1.25M, 2.5M, and 625K tokens/s.
- Activation edges are marked 458 and 687 tokens/s.

**The hundreds, not the millions, set the flow.**
**This is an illustration, not a measured cluster.**




**(a)** A 3-node cluster with model placement. Network connections between the coordinator and compute nodes transmit tokens (4 Byte) while others transmit intermediate activations (16 KB)

**(b)** Graph abstraction of the cluster.

**Figure 2.** Graph abstraction of a 3-node cluster with given model placement. Numbers on the edges in Fig. 2b represent their capacity, which is the number of tokens that can pass through the edges per second. Max flow between source and sink equals the max serving throughput of the cluster.

Source: Fig. 2 annotations. PDF p. 4. Endpoint of each number is the figure's, not a new measurement.

# MILP Variables and Objective

HELIX / MILP

| Symbol | Role | Meaning |
|---|---|---|
| $s_i$ | integer | first layer on node $i$ |
| $b_{ij}$ | binary | node $i$ holds exactly $j$ layers |
| $f_{i,j}$ | real | flow from $i$ to $j$, in tokens/s |
| $d_{i,j}$ | binary | whether link $(i, j)$ is valid |
| *cond* | binary | auxiliaries that linearize overlap |

$e_i = s_i + \sum_j j\, b_{ij}$. Node $i$ holds the half-open range $[s_i, e_i)$. $k$ is the most layers that fit on that GPU.

$$\max \sum_i f_{\text{source},i}$$

**The objective maximizes coordinator outflow. Table 5 groups the variable counts as linear in $|C|$ or $|E|$.**

Source: Sec. 4.4, Tables 5–6. Notation here is compacted from the paper.

# Placement and Memory Constraints

**HELIX / MILP**

$$\sum_{j=1}^{k} b_{ij} = 1, \qquad 0 \le s_i < L, \qquad e_i \le L.$$

- Exactly one layer-count is chosen. The range must lie inside the model's $L$ layers.
- Memory enters as $k$, the maximum number of layers the GPU can hold. Table 6 has no KV-byte variable.
- If $s_j < e_i < e_j$, traffic from $i$ runs only layers $[e_i, e_j)$ on $j$. The paper calls this partial inference.
- The solver may be run with partial inference disabled. Then a link is valid only when $e_i = s_j$.

**GPU memory limits how many layers are placed. KV capacity is enforced later, at schedule time.**

Source: Sec. 4.4, Constraint 1, and the partial-inference remark.

# Flow and Compute Constraints

HELIX / MILP

**Constraint 2: conservation at node $i$.**

$$\sum_{u} f_{u,i} = \sum_{v} f_{i,v}.$$

**Constraint 3: profiled inference throughput.**

$$\sum_{u} f_{u,i} \leq \sum_{j=1}^{k} b_{ij}\, T_j.$$

- $T_j$ is a profiled constant: tokens per second when this node holds $j$ layers.
- The binaries $b_{ij}$ pick exactly one of those profiled rates.
- Flow is still also limited by the network edges on the next slide.

**A node cannot be assigned more token flow than the profiled rate for the layer count it actually holds.**

Source: Sec. 4.4, Constraints 2–3. Sums skip node $i$ itself, as stated in the paper.

# Connectivity and Network Constraints

HELIX / MILP

**Constraint 4, compacted.** A coordinator link into $i$ exists only if $i$ holds the first layer:

$$s_i \le L\,(1 - d_{\text{source},i}). \qquad L\,d_{i,\text{sink}} \le e_i.$$

Between compute nodes, $d_{ij} = 1$ only if $s_j \le e_i < e_j$. Two auxiliary binaries linearize that test, and $d_{ij} \le 0.5\,cond^1 + 0.5\,cond^2$.

**Constraint 5: transmission throughput.**

$$f_{ij} \le d_{ij}\,S_{ij}.$$

$S_{ij}$ is the profiled token rate of that link, from Sec. 4.3. Invalid links carry zero flow.

**Routing variables cannot use a hop that the layer ranges do not justify.**

Source: Sec. 4.4, Constraints 4–5. The two-auxiliary form is the paper's linearization.

# Making the MILP Practical

HELIX / MILP

**PRUNE**

**Average degree 12**

Drop slow links before solving. Sec. 6.8 checks the throughput effect.

**WARM START**

**Heuristic seeds**

Petals, Swarm, and separate pipelines provide the initial solution.

**EARLY STOP**

**Analytic bound**

Stop near the sum of compute rates divided by the layer count.

- Relaxing the integers to an LP is rejected: rounding $s_i$ or $b_{ij}$ can delete edges and change the flow.
- Gurobi stops if 10 minutes pass with no improvement. The budget is 4 hours on a 14-core CPU.
- For hundreds of nodes, the paper only suggests cutting the cluster into smaller pieces first.

**The formulation can certify optimality. The running system also prunes, seeds, and stops early.**

Source: Sec. 4.5–4.6 and the Helix setup in Sec. 6.2.

# Runtime Components and Control Flow



1. **Once per cluster:** profile nodes and links, solve the MILP, load each node's layer range.
2. **On request arrival:** the coordinator's scheduler builds one pipeline and sends the request to its first node.
3. **On each node:** run the assigned layers for the requests that arrived during the previous batch. Do not add a waiting window.
4. **On the last node:** return the output token to the coordinator.
5. **On the next token:** reuse that request's pipeline. Do not solve the MILP again.

**Placement is a cluster constant. The per-token path is a walk on the graph that placement created.**

Source: Sec. 5 and Fig. 3. Batching details are in Sec. 5.1.

# Per-Request Pipelines

HELIX / RUNTIME

- A valid pipeline runs every layer once, in order. Each stage names a node and the layers it will run.
- Petals-style fixed pipelines give each pipeline a disjoint set of machines.
- Helix lets pipelines intersect. The candidates are the source–sink paths of the placement graph.
- After the pipeline is chosen, a node batches every request that arrived while it ran the previous batch. There is no extra collect window.
- Later tokens of the same request keep that pipeline.

**Flexibility comes from many overlapping paths, not from re-solving placement per token.**

Source: Sec. 5.1. Fixed-pipeline contrast cites prior systems, including HexGen in the related work.

# Converting Max-Flow Weights into Scheduling

HELIX / RUNTIME

### IWRR

**Interleaved weighted round-robin**

Each vertex has a scheduler. A neighbor's weight is the max-flow value on that outgoing edge.

### THE WALK

**Coordinator, then each node**

Pick stage 1 from the coordinator. Pick stage 2 from that node's scheduler. Stop when the path is complete.

### WHY INTERLEAVE

**Avoid bursts**

Frequency follows the weight, but choices are interleaved so one heavy edge is not drained in a burst.

### MASK

**Before the walk**

Nodes over the KV high-water mark are removed from the candidate list. Details are on the KV slide.

**The MILP decides the rates. IWRR is the online rule that realizes those rates.**

Source: Sec. 5.1. IWRR follows Tabatabaee et al. as cited by the paper.

# Pipeline Scheduling Example

HELIX / RUNTIME

- Vertices are compute nodes. An edge exists only if the MILP placement made it valid.
- The number on an edge is flow in the max-flow solution, not link capacity.
- The figure's two example pipelines are $1 \rightarrow 5 \rightarrow 9$ and $2 \rightarrow 6 \rightarrow 8$.
- Those paths can share the cluster. They are not two frozen replicas.

**Read the integers as scheduled token rates, then read the paths as two requests.**

Source: Sec. 5.1, Fig. 4. PDF p. 8.

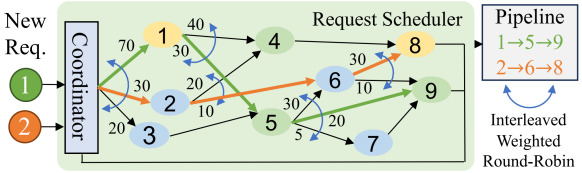



**Figure 4.** Topology graph of a cluster, where each vertex is a compute node, and each edge is a valid network connection. Numbers over edges represent the flow over the network connection in the max flow solution. The pipelines used to schedule requests 1 and 2 are shown on the right.

# KV-Cache Masking and Backpressure

HELIX / RUNTIME

- Each GPU has a finite VRAM budget for KV cache. Exceeding it forces offload to host memory, which the paper says significantly cuts throughput.
- Output length is unknown until generation ends, so the scheduler cannot know the exact KV footprint.
- Helix estimates each node's KV use with the average output length.
- A node above the high-water mark is masked out of IWRR until some of its current requests finish.
- The paper does not print the numeric threshold or the byte formula for that estimate.

**Backpressure is a mask on the same IWRR walk. It is not a second placement solver.**

Source: Sec. 5.2. "High water mark" is the paper's term; its value is not specified.

# Implementation and Prototype

HELIX / EVALUATION

PROTOTYPE

**vLLM 0.4.0.post1**

1.5k lines of Python and 1.7k lines of C++. A unified page pool supports partial inference. ZeroMQ carries inter-node traffic. Gurobi solves the MILP.

SIMULATOR

**14k lines of Python**

It replays heterogeneous GPUs and links. Geo-distributed and 42-node results use this simulator.

FIDELITY

**Single-cluster check**

Against the prototype, decode-throughput error is under 5%. Prompt-latency error is under 5%, decode-latency error under 4%.

SCOPE

**Single-GPU nodes**

Multi-GPU nodes are supported by in-node TP, but the prototype experiments use one GPU per node.

**Trust the simulator inside the single-cluster error above. Do not treat that as a geo-network error bar.**

Source: Sec. 6.1 and Sec. 6.3. FP16 execution is stated in Sec. 6.2.

# Evaluation Methodology



| Setup | Hardware | Network | Where |
|---|---|---|---|
| Single | 4 A100, 8 L4, 12 T4 | 10 Gb/s, one region | Prototype |
| Geo | 4 A100; 2 L4+8 T4; 6 L4+4 T4 | ∼100 Mb/s, ∼50 ms | Simulator |
| 42-node | 7 node types, including V100 and multi-GPU | same simulator stack | Simulator |

- Models: LLaMA-1 30B and LLaMA-2 70B, FP16. Baselines: Swarm, and separate pipelines (SP) inside Helix.
- Petals is compared only as a placement (Sec. 6.6), not as an end-to-end server.
- Offline metric: average decode throughput, in generated tokens/s. Online adds average prompt and decode latency.
- Online arrivals are the Azure shape scaled to 75% of that cluster's peak. Offline arrivals saturate the cluster.

**A 3× on the 42-node simulator is not a 3× on the 24-node prototype.**

Source: Sec. 6.2. The 100 Mb/s geo setting is justified by Table 7, whose pairs span 54–204 Mb/s.

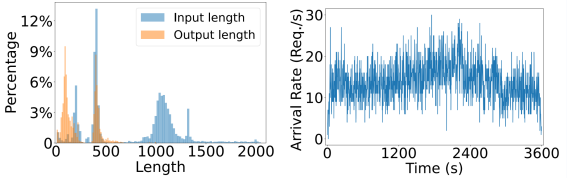
# Workload Characteristics

HELIX / EVALUATION

- Source: Azure Conversation. Requests with input $> 2048$ or output $> 1024$ are removed.
- What remains: 16,657 requests, mean input 763 tokens, mean output 232 tokens.
- Online: 30 s warmup, then 30 min. Offline: 1 min warmup, then 10 min.
- Fig. 5 is the trace's length mix and arrival shape, before the 75% scaling used online.

**Latency ratios below are for this pruned trace, not for short-prompt serving.**

Source: Sec. 6.2, Fig. 5. PDF p. 8.




**(a)** Length distribution. **(b)** Arrival rate.

**Figure 5.** Statistics of Azure Conversation dataset.

# Single-Cluster Results



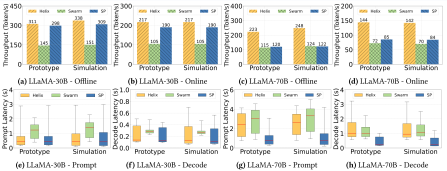



**Figure 6. Single cluster results: (a - d)** decode throughput, **(e - h)** prompt and decode latency for online serving. The box shows the 25 and 75 percentile. The whisker shows the 5 and 95 percentile. The red line shows median.

Fig. 6. Prototype. (a–d) decode throughput; (e–h) online latency. Box 25–75, whisker 5–95, red line = median.

| Model | Versus | Offline tput | Online tput | Prompt lat. | Decode lat. |
|---|---|---|---|---|---|
| 30B | SP | +4% | +14% | −2% | +10% |
| 30B | Swarm | 2.14× | 2.07× | −32% | −12% |
| 70B | SP | 1.86× | 1.69× | average latency higher | |
| 70B | Swarm | 1.94× | 2.00× | −15% | +16% |

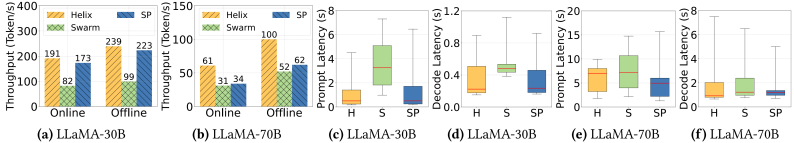
30B: three replicas, one GPU type each. 70B vs. SP: latency is higher because SP serves most requests on A100s and leaves L4/T4 idle.

**Throughput gains are real on the prototype. They are not a uniform latency win.**

Source: Sec. 6.3, Fig. 6. Pairs are offline then online. PDF p. 9.

# Geo-Distributed Results

HELIX / RESULTS




(a) LLaMA-30B (b) LLaMA-70B (c) LLaMA-30B (d) LLaMA-30B (e) LLaMA-70B (f) LLaMA-70B

**Figure 7. Geo-distributed clusters: (a - b)** decode throughput, **(c - f)** prompt and decode latency for online serving. "H" stands for Helix, "S" stands for Swarm. Box: 25 and 75 percentile. Whisker: 5 and 95 percentile. Red line: median.

Fig. 7. Simulator. H = Helix, S = Swarm. Same GPUs as the single cluster, with ∼100 Mb/s between clusters.

**LLaMA 30B vs. SP:** +7% / +10% throughput; prompt −14%, decode latency +2%. **Vs. Swarm:** 2.41× and 2.33× across the two modes; prompt −66%, decode latency −24%.

**LLaMA 70B.** Helix's pipeline is 28% shallower than Swarm, and 19% shallower than Helix's own fast-network plan. Vs. SP: 1.61× / 1.79×. Vs. Swarm: 1.92× / 1.97×; prompt −21%, decode −7%. Swarm's offline prompt latency is 71 s, 7.5× Helix.

**The abstract's "up to 66% and 24%" is this 30B online comparison against Swarm, in simulation.**

Source: Sec. 6.4, Fig. 7. Sec. 6.4 does not label which of 2.41 and 2.33 is online. PDF p. 10.

# High GPU-Heterogeneity Results

HELIX / RESULTS

- Simulator, LLaMA 70B, 42 nodes, 7 types. V100, T4, and 2×T4 cannot form a pipeline alone.
- SP drops those machines. SP+ adds a mixed pipeline that uses them.
- Vs. Swarm / SP / SP+, offline throughput is 1.37×, 2.91×, 2.24×.
- Online: 1.48×, 3.29×, 2.54×. Prompt latency changes by −17%, −1%, −7%.
- Helix's decode latency is slightly higher: baselines leave slow GPUs idle.

**The abstract's 3.3× is the 3.29× online gain versus SP, rounded.**

Source: Sec. 6.5, Fig. 8. PDF p. 10. Not a prototype measurement.

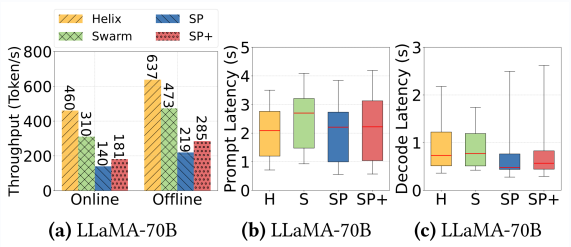
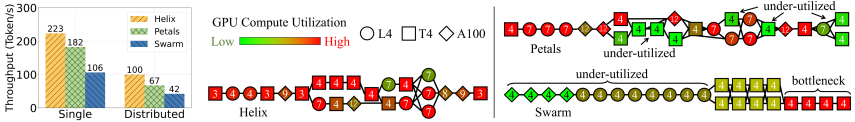



**Figure 8. High GPU-heterogeneity clusters setup: (a)** decode throughput, **(b - c)** prompt and decode latency for online serving. "H" stands for Helix, "S" stands for Swarm.

# Model Placement Deep Dive

## HELIX / ABLATION




**(a)** Decode throughput. **(b)** Model placement case study. Numbers on each node represent the number of layers the node holds.

**Figure 9. (a)** Comparing different model placement methods with offline serving of LLaMA 70B. **(b)** The case study shows model placements for serving LLaMA 70B on the single cluster setup.

Fig. 9. Offline LLaMA 70B. Every method uses Helix's scheduler. Integers in (b) are layer counts.

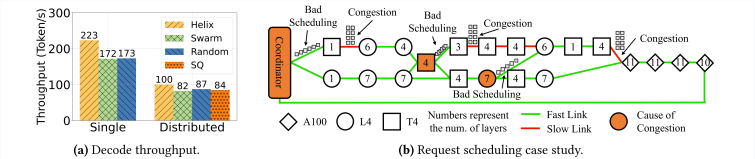
- Vs. Petals / Swarm: 1.23× / 2.10× on the single cluster, and 1.49× / 2.38× on the geo-distributed simulator.
- Single-cluster case: Swarm's tail is four T4s with four layers each, so A100 and L4 stay under-used.
- Petals leaves eight T4s and one L4 under-used. Helix's plan keeps almost every node busy.

**The placement gap remains after the scheduler is held fixed.**

Source: Sec. 6.6, Fig. 9. PDF p. 11. Petals is not an end-to-end baseline.

# Request Scheduling Deep Dive

## HELIX / ABLATION




**(a)** Decode throughput. **(b)** Request scheduling case study.

**Figure 10. (a)** Comparing different request scheduling methods with offline serving of LLaMA 70B. **(b)** A case study that illustrates the congestion in Swarm and random scheduling when serving LLaMA 70B in geo-distributed clusters setup.

Fig. 10. Offline LLaMA 70B. Every method uses Helix's placement. (b) is the geo-distributed case study.

- Vs. Swarm / random: +30% / +29% on the single cluster, and +22% / +15% on the geo-distributed clusters.
- Vs. shortest queue, geo-distributed only: +19% decode throughput.
- In (b), prompt traffic waits 5–16 s on three slow links. One of those queues is caused by a scheduler three hops away.

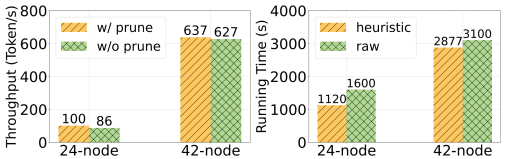
**A local queue rule cannot see a congested cut that starts several hops upstream.**

Source: Sec. 6.7, Fig. 10. PDF p. 12.

# MILP Optimization Ablation

HELIX / ABLATION

- Offline LLaMA 70B. 24-node is the geo setup; 42-node is the high-heterogeneity setup.
- Degree-12 pruning drops 50% and 72% of the links, and 36% and 46% of the problem size.
- With pruning, decode throughput is 16% and 2% higher. The paper attributes this to an easier search under a time limit.
- Warm start finds the same placement, in 43% and 8% less wall time.




**(a)** Cluster pruning. **(b)** Initial values.

**Figure 11.** Ablation study on MILP optimization.

| | With pruning | Without |
|---|---|---|
| 24-node | 876 var, 1122 cstr | 1376 var, 1848 cstr |
| 42-node | 2144 var, 2772 cstr | 4004 var, 5502 cstr |

**Pruning changes the search problem. It is not a proof that the deleted links were useless.**

Source: Sec. 6.8, Fig. 11, Table 8. PDF p. 12. Speedups vary by instance, as the paper notes.

# Solver Time and Solution Quality

**HELIX / ABLATION**

- Instance: LLaMA 30B on 4 L4s and 6 T4s. This is smaller than the 24- and 42-node runs.
- The best feasible placement reaches the optimum in under 5 minutes.
- Certifying that optimum takes more than an hour of branch-and-bound.
- The red dotted line is that cluster's optimal throughput. The upper bound tightens as cuts are added.

**Early stop is reasonable here. Fig. 12 does not certificate the 42-node plan.**

Source: Sec. 6.9, Fig. 12. PDF p. 13.

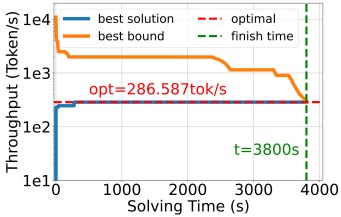



**Figure 12.** Best solution and best upper bound found by the MILP solver relative to solving time. The red dotted line marks the optimal throughput for this cluster.

# Helix vs. ThunderServe, Hetis, and Tessera

**CROSS-PAPER COMPARISON**

| | ThunderServe | Hetis | Helix | Tessera |
|---|---|---|---|---|
| Granularity | Prefill / decode phase | Dense module vs. attention head | Layer range and pipeline | CUDA kernel |
| Decision | Group, phase, route | Primary set, then heads | Placement, then a path | Kernel-to-GPU map |
| Time scale | Offline search + light reschedule | Offline plan + online re-dispatch | Once per cluster + per request | Offline MILP + monitor window |
| State moved | KV at the phase boundary | QKV, attention output, KV shards | Tokens and activations | Cut tensors; KV delta |
| Adaptation | Phase role and failure | Re-dispatch, migration | KV mask; path is reused | Latency / throughput switch |
| Environment | Heterogeneous cloud | Local heterogeneous cluster | Heterogeneous, including geo | Heterogeneous pairs and clusters |
| Models | LLM | LLM | LLM (LLaMA 30B/70B) | LLM, SSM, MLLM, diffusion |
| Main limit | Heuristic phase search | Head groups; local network | One-shot plan; geo is simulated | MILP ignores GPU memory |

**These systems cut different bottlenecks. Their speedups are not a ranking.**

Source: Our synthesis. Helix cells: this PDF. Other cells: each system's own paper, not Helix's related work.

# Limitations and Threats to Validity

HELIX / DISCUSSION

- Placement is computed once. A workload shift does not trigger a new MILP in the described runtime.
- Geo-distributed and 42-node numbers are simulated. The $< 5\%$ error is a single-cluster prototype check.
- The KV mask uses average output length. The paper does not specify the high-water mark.
- Pruning and the 10-minute stall rule mean a deployed plan need not be a proven optimum.
- Decode latency can rise when Helix actually uses the slow GPUs that SP leaves idle.
- Evidence is FP16 LLaMA-1 30B and LLaMA-2 70B on one pruned Azure trace. The conclusion's "guaranteed optimal" is the MILP's claim, not every timed run.

**Report the cluster, the simulator, and the metric together with any ratio.**

Source: Sec. 4.5, Sec. 6, and Sec. 8. The last bullet contrasts the conclusion with Sec. 6.2 and Sec. 6.9.

# Conclusion

HELIX / CONCLUSION

**1**

### A placement is a flow

Edge capacity is tokens per second. Max flow is the throughput of that placement, not its latency.

**2**

### Two clocks

MILP places layers once. IWRR turns the flow into a per-request path and reuses it for later tokens.

**3**

### Where it pays

The large gains appear when one GPU type cannot hold the model, or when the wide-area network is the bottleneck.

- On the reported settings, decode throughput improves by up to 3.29× versus SP (abstract: 3.3×).
- The largest latency cuts versus Swarm, 66% prompt and 24% decode, are the simulated geo-distributed LLaMA 30B online run.

**Helix jointly chooses layer placement and request paths. It does not claim a latency win on every baseline.**

Source: Sec. 8, with the ratios taken from Sec. 6.4 and Sec. 6.5 rather than from the abstract alone.

# Questions & Discussion

Max flow / per-request pipelines / what is simulated

Helix (layer and path) next to phase, head, and kernel systems